

Name: Tridib Banik
Student#: 400514461

Answer to the Question Number 1

The following section outlines the closure computations for various attribute sets and determines whether they qualify as candidate keys. Each test expands the attribute set using functional dependencies until no new attributes can be added. If the closure equals all attributes and the set is minimal, it is a candidate key. Otherwise, it is not a candidate key. The bolded and highlighted (in cyan color) Attribute sets in the tables below are identified as candidate keys.

First, let's consider all attributes individually.

Attribute set	Closure Test and Result
$\{A\}^+$	$\{A\}$ $\{A, B\} (A \rightarrow B)$
$\{B\}^+$	$\{B\}$
$\{C\}^+$	$\{C\}$ $\{A, C, F, H\} (C \rightarrow AFH)$ $\{A, C, F, G, H\} (C \rightarrow ACG)$ $\{A, B, C, F, G, H\} (A \rightarrow B)$ $\{A, B, C, E, F, G, H\} (BC \rightarrow EH)$

	{A, B, C, D, E, F, G, H} (AEH -> BD)
$\{D\}^+$	$\{D\}$
$\{E\}^+$	$\{E\}$ $\{E, F\}$ (E -> F)
$\{F\}^+$	$\{F\}$
$\{G\}^+$	$\{G\}$
$\{H\}^+$	$\{H\}$ $\{A, E, H\}$ (H -> EA) $\{A, B, D, E, H\}$ (AEH -> BD) $\{A, B, D, E, F, G, H\}$ (ABD -> FGH) $\{A, B, C, D, E, F, G, H\}$ (DF -> AC)

Secondly, let's consider all attributes in a pair.

Attribute set	Closure Test and Result
$\{AG\}^+$	$\{A, G\}$ $\{A, B, G\} (A \rightarrow B)$
$\{AF\}^+$	$\{A, F\}$ $\{A, B, F\} (A \rightarrow B)$
$\{AE\}^+$	$\{A, E\}$ $\{A, B, E\} (A \rightarrow B)$ $\{A, B, E, F\} (E \rightarrow F)$
$\{AD\}^+$	$\{A, D\}$ $\{A, B, D\} (A \rightarrow B)$ $\{A, B, D, F, G, H\} (ABD \rightarrow FGH)$ $\{A, B, C, D, E, F, G, H\} (H \rightarrow ABCDEFGH)$
$\{AB\}^+$	$\{A, B\}$
$\{BG\}^+$	$\{B, G\}$
$\{BF\}^+$	$\{B, F\}$
$\{BE\}^+$	$\{B, E\}$ $\{B, E, F\} (E \rightarrow F)$

$\{BD\}^+$	$\{B, D\}$
$\{DG\}^+$	$\{D, G\}$
$\{DF\}^+$	$\{D, F\}$ $\{A, C, D, F\}$ ($DF \rightarrow AC$) $\{A, B, C, D, E, F, G, H\}$ ($H \rightarrow ABCDEFGH$)
$\{DE\}^+$	$\{D, E\}$ $\{D, E, F\}$ ($E \rightarrow F$) $\{B, D, E, F, H\}$ ($DE \rightarrow HB$) $\{A, B, C, D, E, F, G, H\}$ ($H \rightarrow ABCDEFGH$)
$\{EG\}^+$	$\{E, G\}$ $\{E, F, G\}$ ($E \rightarrow F$)
$\{EF\}^+$	$\{E, F\}$
$\{FG\}^+$	$\{F, G\}$

After evaluating all possible combinations, the minimal attribute sets that function as candidate keys are: **C, H, AD, DF, and DE**. Any superset of these keys is not minimal or not candidate keys and therefore excluded.

Answer to the Question Number 2(a)

$RDT \rightarrow P$ is the first FD.

Next, $RDT \rightarrow C$, $C \rightarrow A$ (second and third FDs) becomes $RDT \rightarrow A$ by transitivity.

Next, by Union from Armstrong's axiom, we can combine $RDT \rightarrow A$ and $RDT \rightarrow P$. It becomes $RDT \rightarrow AP$.

So, the FD $RDT \rightarrow AP$ is entailed by F.

Answer to the Question Number 2(b)

Let's compute the closure of RDT, $RDT^+ = RDTPCA$.

How this closure is computed is provided below.

LHS contains RDT. So, add RDT to the closure. Next, $RDT \rightarrow P$. So, add P to the closure. Next, $RDT \rightarrow C$. So, add C to the closure. Finally, $C \rightarrow A$. So, add A to the closure. So, $RDT^+ = RDTPCA$.

Since the closure of RDT contains all of the relation Schedule, it is a key.

Next, let's compute the closure of PDT, $PDT^+ = PDTRCA$.

How this closure is computed is provided below.

LHS contains PDT. So, add PDT to the closure. Next, $PDT \rightarrow R$. So, add R to the closure. Next, $PDT \rightarrow C$. So, add C to the closure. Finally, $C \rightarrow A$. So, add A to the closure. So, $PDT^+ = PDTRCA$.

Since the closure of PDT contains all of the relation Schedule, it is a key.

Next, let's compute the closure of CDT, $CDT^+ = CDT^+PRA$.

How this closure is computed is provided below.

LHS contains CDT. So, add CDT to the closure. Next, $CDT \rightarrow P$. So, add P to the closure. Next, $CDT \rightarrow R$. So, add R to the closure. Finally, CDT itself contains C. It means we can use $C \rightarrow A$. So, add A to the closure. So, $CDT^+ = CDT^+PRA$.

Since the closure of CDT contains all of the relation Schedule, it is a key.

Therefore, all the key(s) of relation Schedule are RDT, PDT, and CDT.

Answer to the Question Number 2(c)

The schedule is not in BCNF. The violating FD is $C \rightarrow A$. Let's decompose it.

$$R = \{C, D, T, R, P, A\}$$

$$C^+ = \{C, A\}$$

$$\text{So, } R_1 = C^+ = \{C, A\}$$

$$R_2 = R - C^+ \cup C = \{C, D, T, R, P, A\} - \{C, A\} \cup \{C\} = \{C, D, T, R, P\}$$

Now, R_1 doesn't violate BCNF (All non-trivial FDs have superkey LHS). So, C is a key for R_1 . So, R_1 is in BCNF.

Next, the FDs that R_2 entails are $RDT \rightarrow P$, $RDT \rightarrow C$, $PDT \rightarrow R$, $PDT \rightarrow C$, $CDT \rightarrow P$ and $CDT \rightarrow R$. (the FD $C \rightarrow A$ no longer applies since A is not in R_2 .).

So, RDT, PDT and CDT are keys of R2. All FDs in R2 have LHS, which is a superkey. So, R2 is in BCNF.

Thus, the decomposition yields two BCNF relations: R1(C,A) and R2(C,D,T,R,P).

Answer to the Question Number 2(d)

The projection occurs onto attributes {D,T,P}.

Closure computations (under the full FD set F)

1. $\emptyset^+ = \emptyset$. Intersection with {D,T,P} = $\emptyset$. $\Rightarrow$ no FD.
2. $\{D\}^+ = \{D\}$ (no FD has LHS = D). Intersection = {D}. $\Rightarrow$ no non-trivial FD from D.
3. $\{T\}^+ = \{T\}$. Intersection = {T}. $\Rightarrow$ no non-trivial FD from T.
4. $\{P\}^+ = \{P\}$. (There is no FD with LHS = P alone.) Intersection = {P}. $\Rightarrow$ no non-trivial FD from P.
5. $\{D, T\}^+ = \{D, T\}$
(RDT $\rightarrow$... and CDT $\rightarrow$... require R or C, not derivable from {D,T}.)
Intersection = {D, T} $\rightarrow$ no non-trivial FD (so DT $\rightarrow$ P does not hold).
6. $\{D, P\}^+ = \{D, P\}$
(PDT $\rightarrow$... requires T.)

Intersection = $\{D, P\} \rightarrow$ no non-trivial FD.

7. $\{T, P\}^+ = \{T, P\}$

(PDT $\rightarrow \dots$ requires D.)

Intersection = $\{T, P\} \rightarrow$ no non-trivial FD.

8. $\{D, T, P\}^+ =$ whole relation (contains D, T, P of course)

Intersection = $\{D, T, P\} \rightarrow$ only trivial FDs from this LHS.

In conclusion, no non-trivial functional dependencies among D, T, and P are implied by the original FD set F. Therefore, the only FDs that hold on ProfsSchedule(D, T, P) are the trivial ones ($X \rightarrow X$ and subsets thereof), which means projected FD set on ProfsSchedule(D,T,P): $\emptyset$ (no non-trivial FDs).

Answer to the Question Number 3(a)

By decomposition,

$A \rightarrow BCD$ (given) becomes $A \rightarrow BC$ and $A \rightarrow D \dots$ (i)

By transitivity,

$A \rightarrow BC$ (from (i)) and $BC \rightarrow DE$ (given) implies $A \rightarrow DE \dots$ (ii)

From (i) and (ii), by union,

$A \rightarrow BC$ and $A \rightarrow DE$ becomes $A \rightarrow BCDE$

By augmentation (with F),

$A \rightarrow BCDE$ becomes $AF \rightarrow BCDEF$

Now, compute the closure based on $AF \rightarrow BCDEF$.

$AF^+ = ABCDEF$

Since the closure of AF contains all attributes of relation R, it is a superkey.

Answer to the Question Number 3(b)

By augmentation (with B), $A \rightarrow D$ (given) becomes $AB \rightarrow DB$...(i)

By decomposition, $AB \rightarrow DB$ (from (i)) becomes $AB \rightarrow D$ and $AB \rightarrow B$...(ii)

By union, $AB \rightarrow C$ (given) and $AB \rightarrow D$ (from (ii)) implies $AB \rightarrow CD$...(iii)

By transitivity, $AB \rightarrow CD$ (from (iii)) and $CD \rightarrow EF$ (given) implies $AB \rightarrow EF$...(iv)

By decomposition, $AB \rightarrow EF$ (from (iv)) can be decomposed to $AB \rightarrow E$ and $AB \rightarrow F$...(v)

From (v), $AB \rightarrow F$.

Answer to the Question Number 4(a)(i)

S1: R1(X); R2(Z); R1(X); R3(X); R3(Y); W1(X); W3(Y); R2(Y); W2(Z); W2(Y)

1: R1(X) R1(X) W1(X)

2: R2(Z) R2(Y) W2(Z) W2(Y)

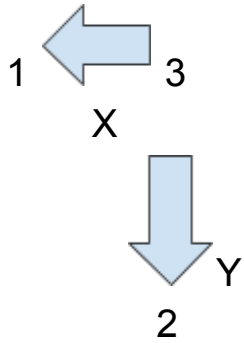
3: R3(X) R3(Y) W3(Y)

W1(X) depends on R3(X).

R2(Y) depends on W3(Y).

W2(Y) depends on R3(Y) and W3(Y).

Precedence graph:



Since there is no cycle in the precedence graph, S1 schedule is conflict-serializable.

The graph requires T3 to come before both T1 and T2, but T1 and T2 are unordered relative to each other.

So the equivalent serial schedules are:

(T3, T1, T2)

(T3, T2, T1)

So, any serial order with T3 first and T1 and T2 in either order is conflict-equivalent to S1.

Answer to the Question Number 4(a)(ii)

S2: R1(X); R2(Z); R3(X); R1(Z); R2(Y); R3(Y); W1(X); W2(Z); W3(Y); W2(Y)

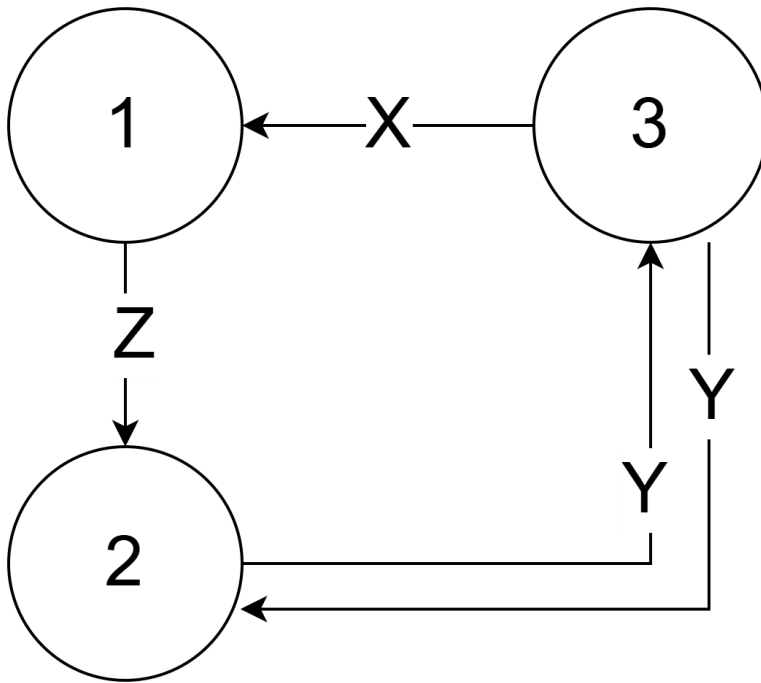
1:	R1(X)		R1(Z)		W1(X)	
2:		R2(Z)		R2(Y)		W2(Z) W2(Y)
3:			R3(X)		R3(Y)	W3(Y)

W1(X) depends on R3(X).

W2(Z) depends on R1(Z).

W3(Y) depends on R2(Y).

W2(Y) depends on W3(Y) and R3(Y).



There is a directed cycle (e.g. $T2 \rightarrow T3 \rightarrow T1 \rightarrow T2$ and $T2 \rightarrow T3 \rightarrow T2$).

The precedence graph contains a cycle, so S2 is not conflict-serializable.

Therefore, no equivalent serial schedule (i.e., no ordering of T1, T2, T3) can produce the same conflict relationships as S2.

Answer to the Question Number 4(b)(i)

S3: R1(X), R1(Y), W1(Y), R2(Y), Commit2, W3(Y), W3(Z), W1(Z), Commit1, Commit3

1: R1(X) R1(Y) W1(Y)

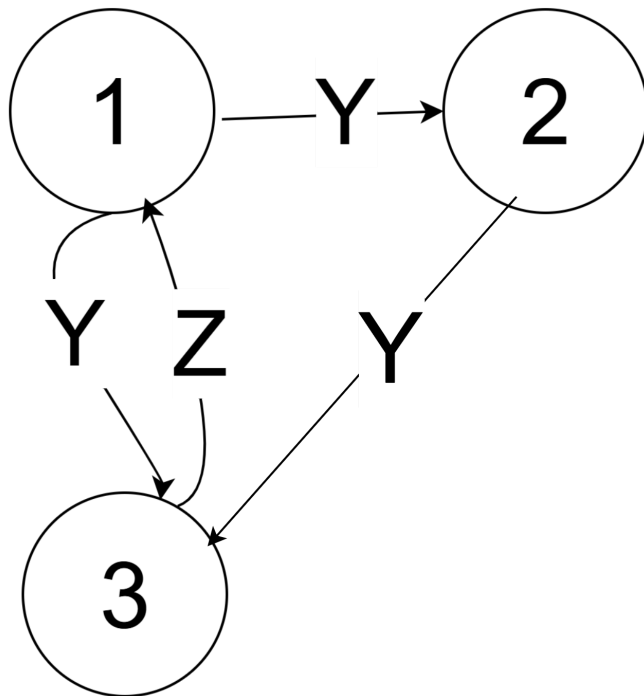
W1(Z) Commit1

2: R2(Y) Commit2

3: W3(Y) W3(Z)

Commit3

The precedence graph for S3 would be:



So, the precedence graph has edges $T1 \rightarrow T3$, $T3 \rightarrow T1$, which creates a cycle between $T1$ and $T3$.

Since there is a cycle $T1 \leftrightarrow T3$, $S3$ is not conflict-serializable.

ACA requires that no transaction reads uncommitted data (i.e., any read must see a committed write). Here, $R2(Y)$ reads the value written by $W1(Y)$ while $T1$ has not yet committed ($T1$ commits later). Therefore, $T2$ reads uncommitted data, which violates ACA. So, $S3$ does not avoid cascading aborts.

Recoverable states that if T_j reads a value written by T_i , then T_i must commit before T_j commits.

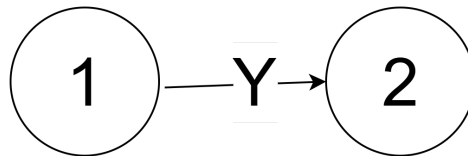
Here $T2$ reads from $T1$ ($R2$ after $W1$) but $Commit2$ occurs before $Commit1$ in the schedule. So, $T2$ committed while depending on an uncommitted $T1$, which violates recoverable. So, $S3$ is not recoverable.

2PL schedules are always conflict-serializable. Since S3 is not conflict-serializable, it cannot be produced by a correct 2PL locking protocol. So, S3 is not 2PL.

Answer to the Question Number 4(b)(ii)

S4: W1(Y), R2(Y), R1(X), R2(X)

1: W1(Y) R1(X)
2: R2(Y) R2(X)



The precedence graph is

Since there is no cycle, it is conflict-serializable. Equivalent serial order: (T1, T2).

ACA requires that reads see only committed data. Here R2(Y) reads the value written by W1(Y) while T1 has not yet committed (commit not shown and assumed later). So, T2 reads uncommitted data, which violates ACA. So, it does not avoid cascading aborts.

Recoverability depends on commit order. Because commits are not shown, we only know commits happen after the listed actions. For recoverability we need T1 to commit before T2 (since T2 reads T1's write). That might happen (if Commit1 happens before Commit2), or it might not (if Commit2 happens first). The schedule as given does not fix commit order, so we cannot determine whether the recoverability property holds. So, it is undetermined (could be recoverable if Commit1 precedes Commit2, otherwise not).

Determining 2PL requires knowledge of locking/unlocking events (which are not shown). While every 2PL schedule is conflict-serializable, the converse need not be true; conflict-serializability alone does not prove a schedule was produced by 2PL. With only the operation order and no lock/unlock info, we cannot determine whether this schedule follows 2PL. So, it is undetermined.

Answer to the Question Number 5

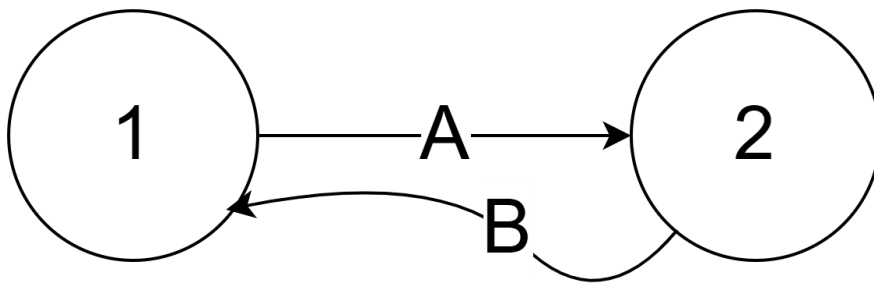
In this specific locking protocol, since shared locks are released immediately after reads, the schedule does not follow two-phase locking (2PL). Without strict lock holding, operations may interleave and create cycles in the precedence graph. So, this protocol might violate conflict-serializable.

For example,

T1	T2
X(A)	
	X(B)
X(B) -> queued	
	X(A) -> queued

T1 is waiting for T2 to release its lock

T2 is waiting for T1 to release its lock



Since there is a cycle in the precedence graph, it is not conflict-serializable.

Next, recoverability depends on commit order (i.e., if T_i depends on T_j , T_j must commit before T_i), and this protocol does not enforce commit dependencies. So, we cannot guarantee recoverability.

Thirdly, since uncommitted writes cannot be read (writer holds X-lock until commit), if a writer aborts, no other transaction needs to cascade an abort. Basically, readers cannot access uncommitted writes due to the exclusive lock rule, which prevents reading uncommitted data. So, ACA is ensured by this locking protocol.

Transactions may still wait for each other's exclusive locks, forming a circular wait situation. Therefore, deadlocks are still possible.

For example,

T1	T2
X(A)	
	X(B)
X(B) -> queued	
	X(A) -> queued

T1 is waiting for T2 to release its lock

T2 is waiting for T1 to release its lock

Such a cycle of transactions is a deadlock.